

Machine Learning

- Machine learning is a way of teaching computers to learn from data - just we humans learn from experience.
- Instead of programming every step, we give machines lots of data and let them figure out patterns on their own.

Ex:-

- Youtube Recommendations system:- Learn from your watch history.
- Spam Detection in Gmail:- Learning patterns in spam emails.
- Voice Assistants (Siri, Alexa):- Learn how you speak.
- Self driving Cars:- Learn to identify stop signs, pedestrians.
- Face Unlock on Phones:- Learning to recognize your face.

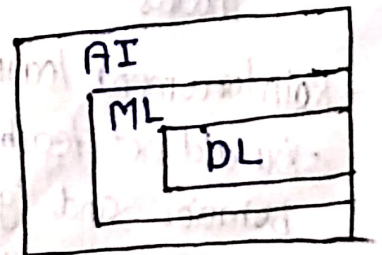
Traditional Programming.

- Give Rule + data $\rightarrow$ result
- manual logic writing

Machine Learning.

- Give data + result $\rightarrow$ get rule (model)
- Learn pattern automatically

- Learning ML means learning the language of the feature.
- Now that you know what Machine Learning is, let's drive into the difference between ML/DL/AI.
- AI:- Artificial Intelligence is the big umbrella - it's the science of making machine.
- ML:- Machine Learning is a subset of AI - this is where machines learn from data and improve over time.
- DL:- Deep learning is the sub set of ML - it uses somethings called neural networks which are inspired by the human brain.



Types of ML

• Machine Learning has 3 major types.

- Supervised Learning.
- Unsupervised Learning.
- Reinforcement Learning.

• Supervised Learning —: is like a student being taught by a teacher.

• We give the machine ~~in~~ Labeled data (input + correct ~~and~~ answer) and it learns to predict.

• But the training of a model are done using various methods and these methods are known as algorithms.

- Linear algorithm
- Logistic Regression
- Naive Bayes
- Support Vector machine
- K-Nearest Neighbours
- Gradient Boosting

Income	Credit Score	Loan
40000	750	Yes
25000	600	No
50000	800	Yes
30000	500	No
60000	950	Yes

• Unsupervised Learning —: Here also we will get the data but we are not going to predict anything. We will just find patterns. Similar Group items, Reduce complexity, spot outliers.

• Unsupervised learning are used with supervised learning to make models.

• Reinforcement Learning —: Reinforcement learning is like training a dog. You did not teach it everything directly. Instead of You reward good behavior and ignore or punish bad actions. Over a time the dog learns what action give it treats that's the core idea of RL - learning by trial and error.

EDA

Let's see the steps involved in making a machine learning model.

① Problem definition

② Data Collection

③ Exploratory Data Analysis (EDA)

④ Data Preprocessing / cleaning.

⑤ Feature Selection and Engineering

⑥ Split the data set.

⑦ Model Selection

⑧ Model Training

⑨ Model Evaluation

⑩ Hyperparameter Tuning

⑪ Model Testing / Validation.

• EDA stands for Exploratory data Analysis - it's the step where you explore the data to:

- Understand it.
- Discover patterns.
- Spot anomalies.
- Generate insights.
- and decide what to do next.

• Think of EDA as detective work - you're looking at your data with curiosity before building any models or doing fancy transformations

EDA Steps

① Viewing the Data.

- `head()`, `tail()`, `shape`, `info()`, `describe()`.
- What columns do I have? What type of data.

② Summary Statistics.

- `mean()`, `median()`, `mode()`, `std()`, `min()`, `max()`, `quantiles()`.
- Help understand spread and central tendency.

③ Value Counts.

- How many unique values in a column? `unique()`.
- Count of categorical columns.

④ Missing Value Analysis.

- Where are the gaps? What percent of data is missing.
- `isnull()`, `isnull().sum()`,

⑤ Fill missing value.

- `fillna()`

⑥ Visualizations.

- Histograms —: distribution of values.
- Box plots —: outliers and spread
- Bar plots —: Comparisons of categories.
- Correlation heatmaps —: linear relation between numerical features.
- Scatter plots —: bivariate relationships.

⑦ Target Variable Exploration.

- How does your output (like 'charges') relate to other variables.

EDA Importance

- You cannot clean or preprocess what you do not understand.
- It helps you identify mistakes, biases or limitations in the data.
- It guides the direction of your data cleaning and feature engg.
- It gives you audience or stakeholders a 'story' or overview of what the data is telling you.

Data cleaning

① Handle Missing values

- Which columns have missing values (nulls)
- Strategies to handle:
 - Drop missing rows/columns (only if very few).
 - Impute with:
 - mean/median —: for numerical data.
 - mode —: for categorical data.
 - ~~advance~~

② Remove Duplicates —:

- Detect and drop exact duplicate rows.

③ Fix Data Types

- convert wrong types (e.g. numbers stored as string, data of kdt)

④ Handle Inconsistent Categories:-

- Clean up categorical value like:
 - 'Male', 'male', 'MALE' → should all become 'male'.
 - 'Yes', 'YES', 'y' → unify to one format.

⑤ Detect and Handle Outliers:

- Use boxplots, IQR, or Z-score.
- Handle by:
 - Removing - if clearly wrong
 - Capping - (e.g. to 95th percentile)

⑥ Fix Logic or Domain Error

Ex:- age = -5, BMI = 150 → Invalid.

- Can replace with mean, median, remove.

You can say:

EDA tells you what's wrong. Data cleaning fixes it. Cleaning is not glamorous, but it's 80% of the work in real world project.

Data Preprocessing

- To prepare clean data so it can be analyzed or used in machine learning model.
- If data cleaning about fixing mistakes, Data preprocessing is about transforming valid data into a usable format.

① Encoding categorical variables:

- Convert text labels (like 'male', 'yes') into numbers.
- one hot encoding → [1, 0, 0], [0, 1, 0], [0, 0, 1] → (non ordered)
- Label encoding → [0, 1, 2] → (ordinal)

② Feature Transformations (log, square root, etc)

- used to handle skewed data, like right skewed or left skewed data.

③ Feature Scaling (Normalization or Standardization).

- Bring numerical values to the same scale especially useful for distance based algorithm.
- Normalization (min-max scaling)
 - scales values b/w $\rightarrow 0$ and 1

$$x_{\text{new}} = \frac{x_i - \min(x)}{\max(x) - \min(x)}$$

- Standardization (Z-score scaling)
 - Transforms data to have mean=0 and std=1.

$$x_{\text{new}} = \frac{x_i - x_{\text{mean}}}{\text{std}}$$

- Creating new features or transforming existing ones to expose useful patterns that ML models can learn from.

Feature Engineering

- Common Feature Engineering Technique in ML:

- Mathematical combinations
- Target Based flags
- Binning (when it helps)
- Time based feature. (if time exists)

- Selecting the most useful features and removing the rest.

- Why is important?

- Reduce noise and over fitting

- Speeds up training

- Improves models accuracy.

- Makes model interpretation easier.

- ① Filter Methods (Pure Statistics)
- Correlation matrix → Remove highly correlated features
 - Chi-square test → (categorical vs categorical)
 - ANOVA F-test (numerical vs categorical target)

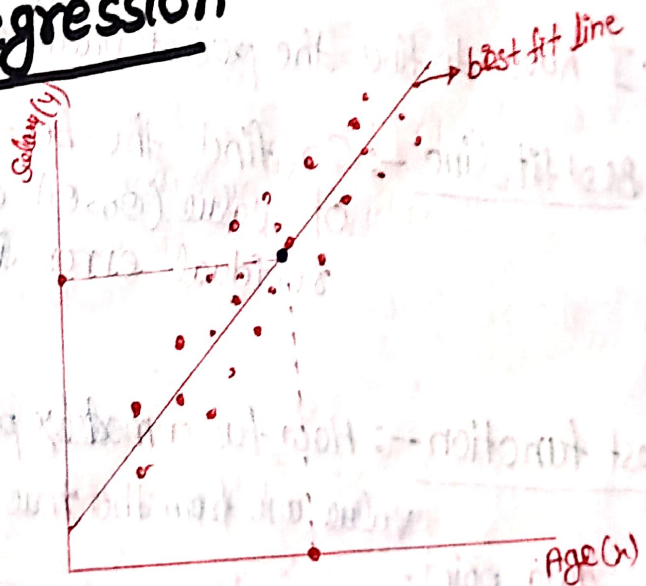
- ② Embedded Method (selection built into model)
- Lasso Regression → shrink coefficients to 0
 - Tree based models → (random forest, XGBoost)

Regression

- Regression is way to predict a number. It helps us find out how one thing changes when something else changes.
- Regression is about finding patterns in numbers so we can make smart prediction.

Linear Regression

Age	Salary
22	36000
26	38000
28	40000
34	40000
42	52000
44	56000



- Now let's see how you actually create the best line.

$$y = mx + b$$

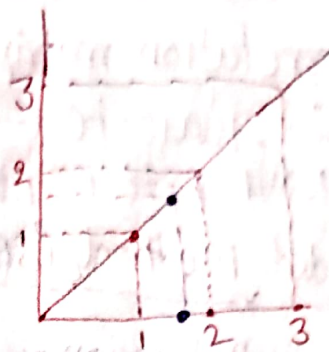
- m → slope
- b → Intercept
- x → data point
- y → Prediction

Case 1 :- $m=1, b=0$

we can calculate if $x=1.5$

$$y=x$$

also $y=1.5$ prediction



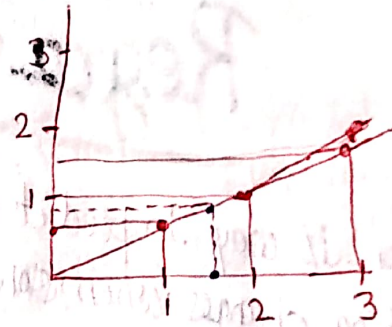
Case 2: $m=0.5, b=0$

We can calculate if $x=1.5$

$$y=mx+b$$

$$y=0.5 \times 1.5 + 0$$

$$y=0.75$$



$$y = b + mx$$
$$h\theta(x) = \theta_0 + \theta_1 x$$

θ_0 = Intercept

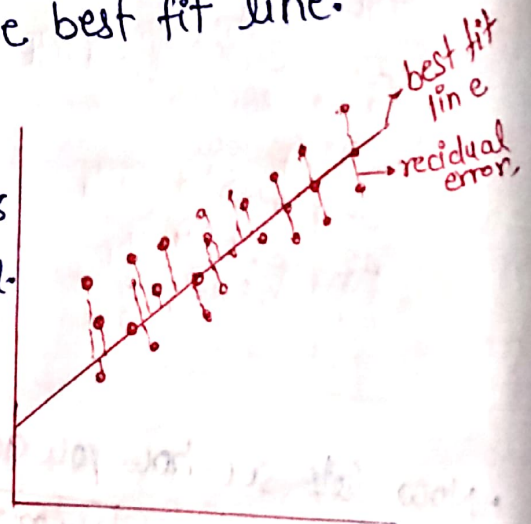
θ_1 = slope

• I have to find the perfect value of θ_0 and θ_1

Best fit line :- To find the best fit line by using the changing of value (θ_0, θ_1) and which gave you less residual error is the best fit line.

Cost function :- How far a model's predicted value are from the true value data point.

$$J(\theta_0, \theta_1) = \sum_{i=1}^m \frac{1}{2m} (h\theta(x^{(i)}) - y^{(i)})^2$$



• You have to minimize the cost function by changing the value of θ_0, θ_1 .

Gradient Descent

{1,1}
{2,2}
{3,3}

assume that

$$\theta_0 = 0, \theta_1 = 1$$

$$h(x) = \theta_0 + \theta_1 x$$

$$h(1) = 0 + 1(1) = 1$$

$$h(2) = 0 + 1(2) = 2$$

$$h(3) = 0 + 1(3) = 3$$



$$\text{Cost function } J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)})^2$$

$$= \frac{1}{2 \times 3} [(1-1)^2 + (2-2)^2 + (3-3)^2]$$

$$= \frac{1}{6} \times 0 = 0 \Rightarrow J(\theta_1), \theta_1 = \{0, 1\}$$

assume, $\theta_0 = 0, \theta_1 = 0.5$

$$h(x) = \theta_0 + \theta_1 x$$

$$h(1) = 0 + 0.5(1) = 0.5$$

$$h(2) = 0 + 0.5(2) = 1.0$$

$$h(3) = 0 + 0.5(3) = 1.5$$

$$\text{Cost function } J(\theta_1), \theta_1 = \{0.5, 0.5\}$$

$$= \frac{1}{2 \times 3} [(1-0.5)^2 + (2-1)^2 + (3-1.5)^2]$$

$$= 0.58$$

assume $\theta_0 = 0, \theta_1 = 0$

$$h(x) = \theta_0 + \theta_1 x$$

$$h(1) = 0$$

$$h(2) = 0$$

$$h(3) = 0$$

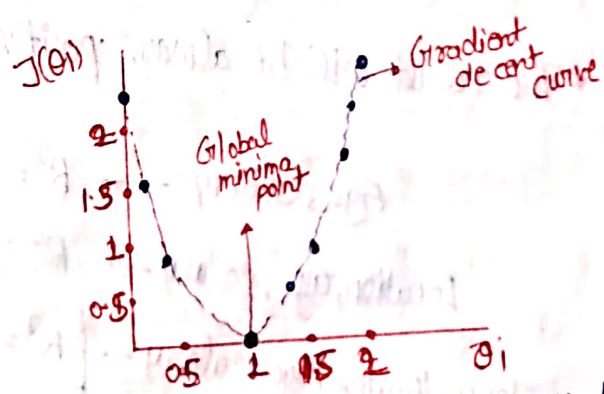
$$\text{Cost function } J(\theta_1), \theta_1 = \{0, 0\}$$

$$= \frac{1}{2} [(1-0)^2 + (2-0)^2 + (3-0)^2]$$

$$= 2.5$$

Now make graph using the Cost-function value.

- Gradient Descent ~~curve~~ is made by the Cost-function.
- It always make in 3D plane.



Repeat Convergent theorem:- Convergence in Gradient Descent means that after repeatedly updating the model parameters the algorithm gradually reaches or get very close to the Global minima point.

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \frac{\partial J(\theta)}{\partial \theta}$$

$[\alpha = \text{learning rate} = 0.05]$

- For the multiple input feature:

$$N(x) = \theta_0 + \theta_1 x$$

$$N(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \theta_4 x_4 + \dots + \theta_n x_n$$

- In the we make the n-dimension vector.
- How to split the data for the training and testing.

80%	→ training data = X_{train}, y_{train}
20	→ testing data = X_{test}, y_{test}

$$X_{train}, X_{test}, y_{train}, y_{test} = \text{train_test_split}(X, y, \text{test_size}=20, \text{random_state}=42)$$

Performance matrix

- ① R^2 ② Adjusted R^2

$$R^2 = 1 - \frac{\text{Sum of Residuals}}{\text{Sum of total}}$$

$$1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

y_i = actual point
 $\hat{y}_i$ = Predicted point
 $\bar{y}$ = mean

- The result will be always positive and the result will be decimal.

Age, salary → $R^2 = 81\%$, $\text{Adj } R^2 = 79\%$

Location, age, salary → $R^2 = 88\%$, $\text{adj } R^2 = 89\%$

sleep, location, age, salary → $[R^2 = 89\% \uparrow, \text{adj } R^2 = 86\% \downarrow]$

$$\text{Adjusted } R^2 = \frac{1 - (1 - R^2)(n - 1)}{N - P - 1}$$

n = number of rows.
 P = features

- In the last $R^2 \uparrow$ but $\text{adj } R^2 \downarrow$ so donot take the sleep column as input.



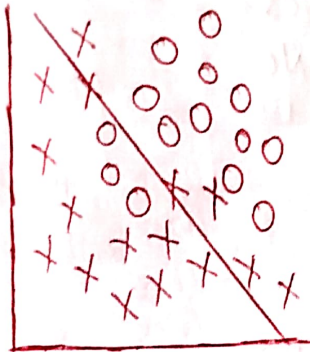
Overfitting & Underfitting

Overfitting:-

training $\rightarrow$ model perform better $\rightarrow$ low bias
testing $\rightarrow$ not model perform good $\rightarrow$ high variance.

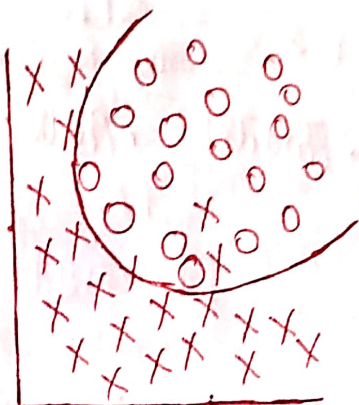
Underfitting:-

training $\rightarrow$ model perform bad $\rightarrow$ high bias
testing $\rightarrow$ model not perform good $\rightarrow$ high variance.



under fitting

Training $\rightarrow$ 68%
Testing $\rightarrow$ 64%



Appropriate fitting

Training $\rightarrow$ 92%
Testing $\rightarrow$ 90%



Overfitting.

Training $\rightarrow$ 96%
Testing $\rightarrow$ 64%

Ridge & Lasso Regression

- Both are used in overfitting condition.
- Regularization adds penalty term to the loss function.

Ridge Regression

- Ridge Regression use L2 Regularization.

$$\text{Cost} = \text{MSE} + \lambda \sum_{j=1}^n \beta_j^2$$

- MSE = Mean Squared Error
- λ = Regularization parameter.
- β_j = model coefficient
- Ridge Regression add the square of the coefficients as a penalty.

Lasso Regression

- Lasso Regression use L1 Regularization.

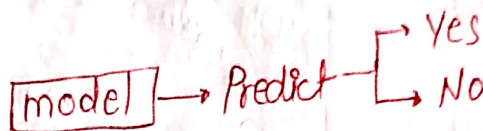
$$\text{Cost} = \text{MSE} + \lambda \sum_{j=1}^n |\beta_j|$$

- adds the absolute value of coefficients as penalty.

Classification

- A classification algorithm is a type of machine learning algorithm used to predict a category or class label for given input.
- "Classification is the process of teaching a machine to answer questions like 'Is this a cat or a dog?' or 'Will this customer survive or not?'"

Weight	obesity
29	No
39	No
48	No
96	Yes
128	Yes
82	Yes

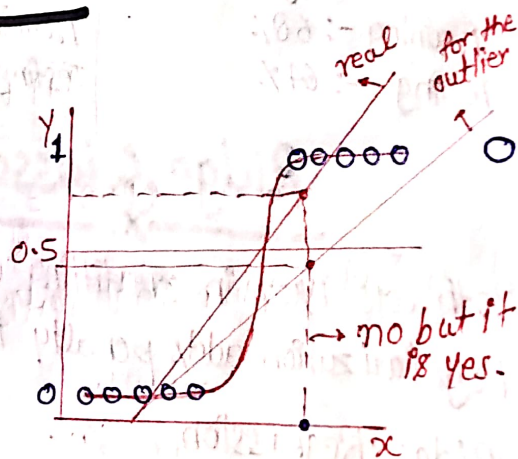


Logistic Regression

- In the classification we did not use the linear regression because.
- ① outlier → Influence the straight line.
- ② Result can be positive or negative.

- So prevent all of these things we squash the line.

- to make the straight line into S shape curve we will use **SIGMOID** activation function.



$$h\theta(x) = g(\theta_0 + \theta_1 x_1)$$

$$g = \frac{1}{1 + e^{-z}}$$

$$h\theta(x) = \frac{1}{1 + e^{-(\theta_0 + \theta_1 x_1)}}$$

e = euler's number = 2.71828

g = sigmoid activation function.

e → give smooth curve

Linear Regression

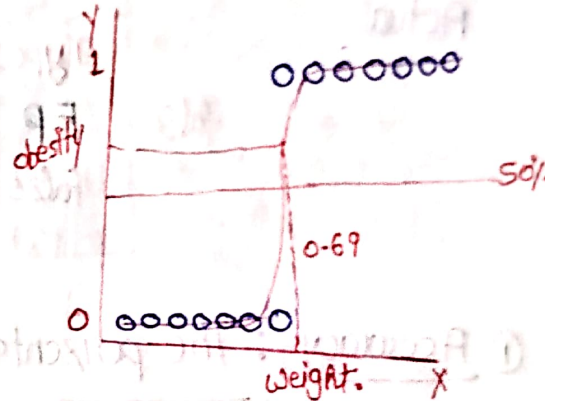
- $\theta_0, \theta_1 \rightarrow$ create the best fit line
- Cost function to find the residual errors.
- output $\rightarrow$ continuous number

Logistic Regression

- $\theta_0, \theta_1 \rightarrow$ create the best squashed line.
- Log loss function to find the residual error.
- output = class $\rightarrow (0,1)$ (Yes, No)

Log loss (Binary cross entropy)

- let's say Predicted probability = 0.99 and actual is 1.
 $\Rightarrow$ very low loss.
- let's say Predicted probability = 0.01 and actual is 1.
 $\Rightarrow$ very high loss.
- If the loss is very high the log loss function gives penalty.



Log loss = Binary cross Entropy.

$$\text{Log loss} = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(\hat{y}^{(i)}) + (1-y^{(i)}) \log(1-\hat{y}^{(i)})]$$

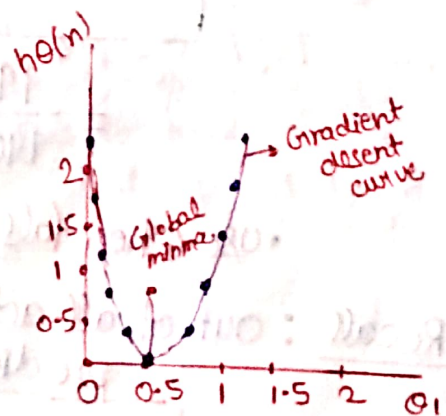
Let assume $y^{(i)} = 1$
 $\hat{y}^{(i)} = 0.54$

$$\text{log loss} = 0.616$$

m = number of sample
 $y^{(i)}$ = actual level (0,1)
 $\hat{y}^{(i)}$ = Predict probability (0-1)

Note:- Log loss and cost function are use to make the gradient descent curve.

- And using of gradient descent curve we find the global minima which is use for make the best fit line or best squashed line.



Confusion matrix

- Confusion matrix is use for evaluate the model.

		Predicted			
		Positive	negative		
Actual	True	TP True positive	Type I error FN False Negative false	190	10
	false	Type II error FP false Positive	TN True negative	10	40

- ① Accuracy:- The percentage of correct Prediction.

$$\text{Accuracy} = \frac{TP + TN}{TP + FN + FP + TN}$$

$$= \frac{190 + 40}{190 + 10 + 10 + 40} = 0.92$$

- ② Precision:- out of all predicted positive how many actual positive.

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$= \frac{190}{190 + 10} = 0.95\%$$

- use when false positive are costly. (spam detection)

- ③ Recall:- out of all actual positive, how many did we correctly predicted.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- use when false negative are costly. (diseases detection).

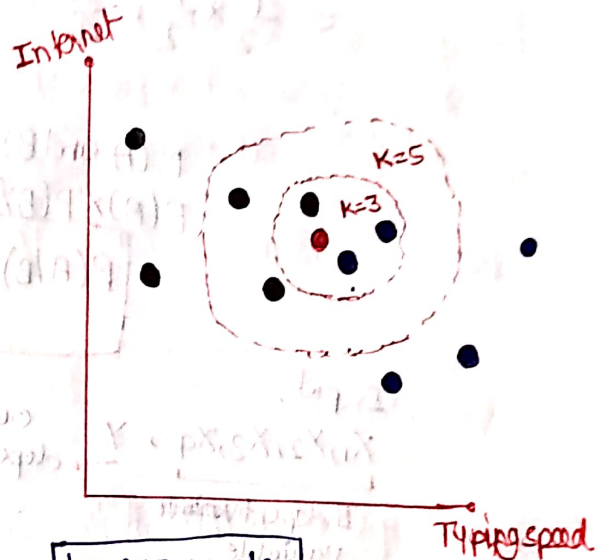
$$= \frac{190}{190 + 10} = 0.95\%$$

④ F1-Score :- Harmonic mean of precision and Recall.

$$F1 \text{ Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

K-Nearest Neighbour

Internet	Typing Speed	Hacker.
24	56	1
28	55	1
12	22	0
10	21	0
⋮	⋮	⋮



① Define K value $\Rightarrow K = \{3, 5, 7, 9\}$ → hyperparameter

② Calculate the distance from all point. (Euclidean distance)

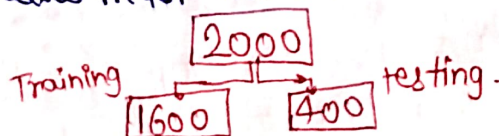
③ Sort all the distance.

④ Majority count.

Ex:- $K=3 \rightarrow 1 \text{ No}, 2 \text{ Yes} \rightarrow \text{Yes}$
 $K=5 \rightarrow 3 \text{ No}, 2 \text{ Yes} \rightarrow \text{No}$

other the classification $\rightarrow$ calculate the mean of K-nearest neighbour.

- You can use in N dimensions.
- K value must be odd number.



$K=3 \Rightarrow 86\%$
 $K=5 \Rightarrow 89\%$
 $K=7 \Rightarrow 90\%$
 $K=9 \Rightarrow 92\%$
 $\vdots$

Naive Bayes

Independent Event

$$\text{Coin} = P(H) = \frac{1}{2}, P(T) = \frac{1}{2}$$

$$\text{Dice} = P(6) = \frac{1}{6}$$

$$P(A \text{ and } B) = P(A) \times P(B) \\ = \frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$$

Dependent Event

Deck of Card = $\begin{array}{l} 52 \\ \swarrow \searrow \\ 26 \quad 26 \\ \text{Red} \quad \text{Black} \end{array}$

$$P(R) = \frac{26}{52} = \frac{1}{2} \text{ if take}$$

$$P(R) = \frac{25}{51}$$

$$P(A \text{ and } B) = P(A) \times P(B/A)$$

$$P(A \text{ and } B) = P(B \text{ and } A)$$

$$P(A) \times P(B/A) = P(B) \times P(A/B)$$

$$P(A/B) = \frac{P(A) \times P(B/A)}{P(B)} \rightarrow \text{Bayes Theorem.}$$

Input.

X_1, X_2, X_3, X_4, Y output
independent variable $\rightarrow$ dependent variable

Independent Variable

$y \rightarrow \text{Yes}$
 $N \rightarrow \text{No}$

$$P(A/B) = \frac{P(A) \times P(B/A)}{P(B)}$$

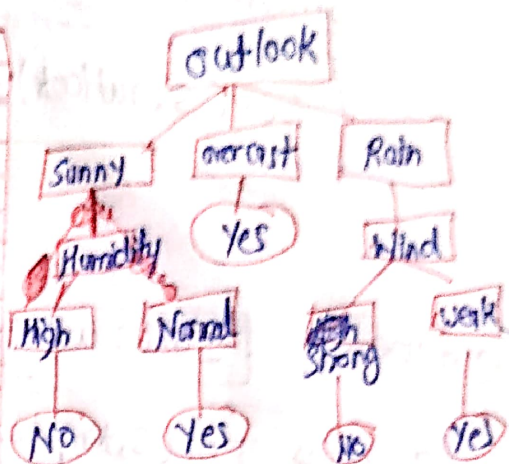
$$P(Y/X_1, X_2, X_3, X_4) = \frac{P(Y) \times P(X_1, X_2, X_3, X_4/Y)}{P(X_1, X_2, X_3, X_4)}$$

$$\text{Yes} \Rightarrow P(Y/X_1, X_2, X_3, X_4) = \frac{P(Y) \times P(X_1/Y) \times P(X_2/Y) \times P(X_3/Y) \dots P(X_n/Y)}{P(X_1) \times P(X_2) \times P(X_3) \dots P(X_n)}$$

$$\text{No} \Rightarrow P(N/X_1, X_2, X_3, X_n) = \frac{P(N) \times P(X_1/N) \times P(X_2/N) \times P(X_3/N) \dots P(X_n/N)}{P(X_1) \times P(X_2) \times P(X_3) \dots P(X_n)}$$

Decision Tree

Outlook	Temperature	Humidity	Wind	play Tennis
Sunny	Hot	High	False	No
Sunny	Hot	High	True	No
overcast	Hot	High	False	Yes
Rainy	Mild	High	False	Yes
Rainy	Cool	Normal	False	Yes
overcast	Cool	Normal	True	No
Sunny	Cool	Normal	True	Yes
Sunny	Mild	Normal	False	No
Rainy	Cool	Normal	False	Yes
Rainy	Mild	Normal	False	Yes
Sunny	Mild	Normal	True	Yes
overcast	Mild	High	True	Yes
overcast	Hot	Normal	False	Yes
Rainy	Mild	High	True	No



• We find the root node using Entropy and Information gain.

- ① Start with the Dataset
- ② Choose the best feature
- ③ make branches
- ④ Repeat the process after each branch.
- ⑤ Stop when:
 - All data is pure
 - max depth is reached.

$$\text{Entropy}(S) = -P_+ \log_2(P_+) - P_- \log_2(P_-)$$

output

play → 5 No, 9 Yes

$$\Rightarrow S = -\frac{9}{14} \left(\log_2 \left(\frac{9}{14} \right) \right) - \frac{5}{14} \log_2 \left(\frac{5}{14} \right)$$

$$S = 0.940$$

Outlook →

- Sunny → 2Y, 3N $S = -\frac{2}{5} \log_2 \left(\frac{2}{5} \right) - \frac{3}{5} \log_2 \left(\frac{3}{5} \right) = 0.971$
- overcast → 4Y, 0N $S = 0$
- Rain = $S = 0.971$

Information Gain =

$$IG(S, outlook) = \text{entropy}(S) - \sum_{\text{output}} \frac{|S_v|}{|S|} \cdot \text{Entropy}(|S_v|)$$

$$= 0.940 - \left[\frac{5}{14} \times 0.975 + \frac{2}{14} \times 0 + \frac{5}{14} \times 0.971 \right]$$

outlook $\Rightarrow$ $IG(S, outlook) = 0.246$

Temp $\Rightarrow$ $IG(S, temp) = 0.029$

humidity $= IG(S, hum) = 0.191$

wind $= IG(S, wind) = 0.048$

- Which Information Gain is greater. It become root node.

Support Vector Machine

Two type of hyper plane

- ① 2D hyper plane
- ② 3D hyper plane.

$$wx + b = 0$$

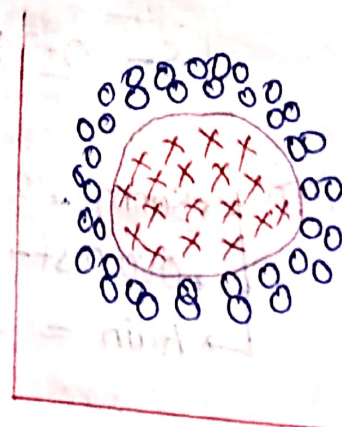
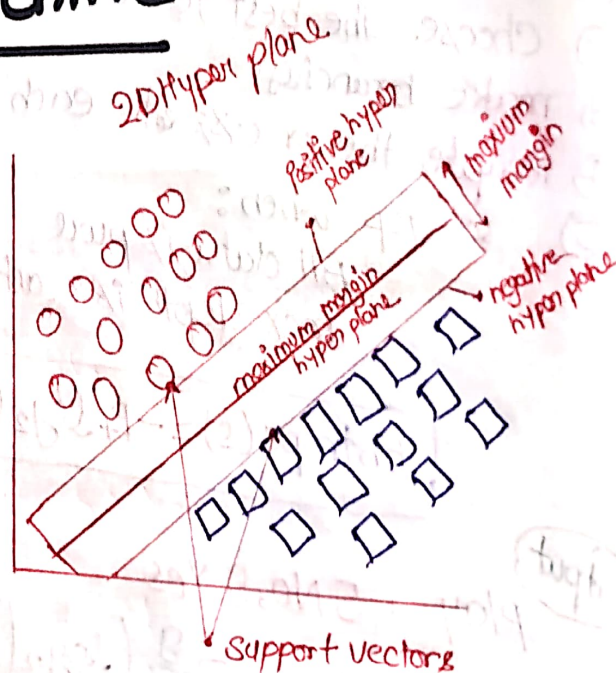
x = point you want to predict

w = weight/slope

b = bias

$$wx + b > 0 \rightarrow \text{Class 1} \rightarrow \bigcirc$$

$$wx + b < 0 \rightarrow \text{Class 2} \rightarrow \square$$



• hyper plane — A line that separate two categories.

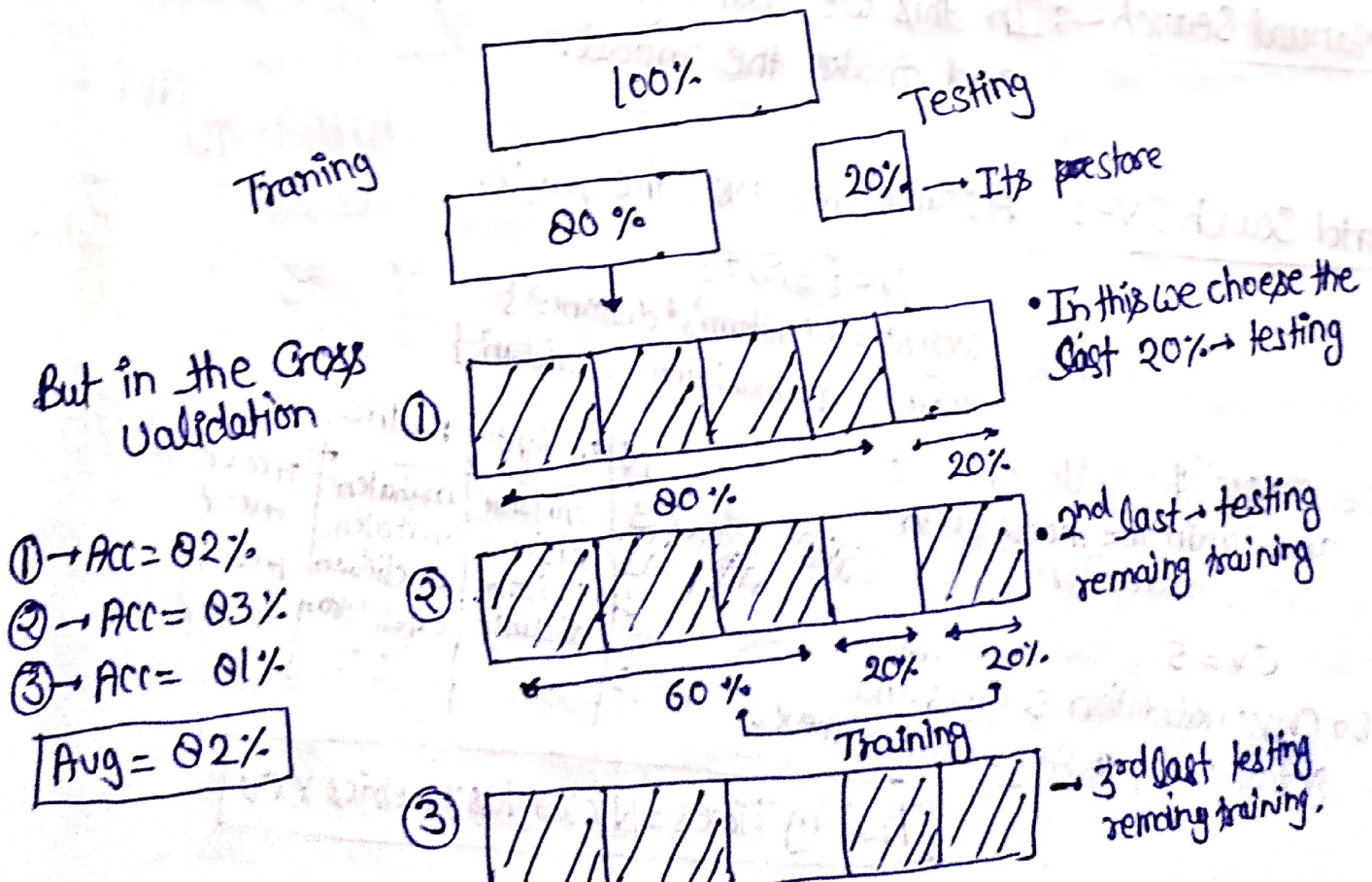
• Margin — The space b/w the line and nearest point

• Support vector — The closest ball to margin that support the line

Model Tuning

- The first version of your model is never the best version. That's where the model tuning comes in.
 - When we train a model, it has certain settings - these are called hyperparameters. The hyperparameters decide how your model behaves - like how deep should a tree be, how many neighbours in KNN, what learning rate we use.
 - But if we just ^{use} random hyperparameters - our model must underfit, overfit.
 - So the goal of model tuning is:
 - ① To find the best combination of hyperparameters.
 - ② So that our model performs better on new, unseen data.
- In short —: Tuning helps you squeeze out the best possible performance from your model and that's why it is an essential step before finalizing any ML model. And so called hyperparameter tuning.

Cross Validation



- You divide the data in k part and every part will be your training data and remains your testing data.

Hyper Parameter Tuning

- The process of find the best configuration of settings for a ML model to improve its accuracy and performance on new data.

- Ridge & lasso Regression $\rightarrow \alpha$ learning rate
- KNN $\Rightarrow n$ -neighbours = $\{3, 5, 7, 9, \dots\}$
- Decision Tree $\Rightarrow$ max depth, sample split, max feature.
- SVM = kernel()

- Method of Hyper Parameters Tuning:

- ① Manual Search
- ② Grid Search CV.
- ③ Random Search CV.

- Manual Search $\rightarrow$ In this we manually takes the hyper parameters and make the model.

- Grid Search CV $\rightarrow$ Assume we use the KNN.

$N = \{3, 5, 7\}$
 $\text{weight} = \{\text{'uniform'}, \text{'distance'}\}$
 $\text{metric} = \{\text{'mahalan'}, \text{'euclidean'}\}$

- We choose the grids one by one and train the model with cross validation.

CV = 5

- Do cross validation 5 times and get accuracy $\Rightarrow$ Avg $\Rightarrow$ every times.

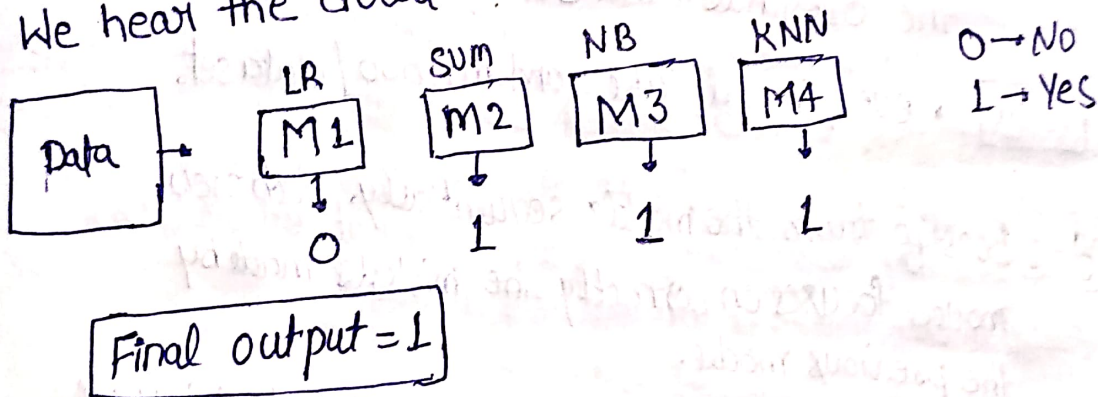
N	weight	metric	Acc = ?
3	uniform	mahalan	Acc = ?
3	distance	mahalan	Acc = ?
3	uniform	euclidean	Acc = ?
3	distance	euclidean	Acc = ?
5	...	...	...

$$\text{Training Times} = N \times \text{weights} \times \text{metrics} \times \text{CV}$$

- Which hyper parameters gives you best accuracy take that combination.
- Random search CV $\rightarrow$ In the Random search CV we choose the hyper parameters Randomly.
- In the grid ~~search~~ search CV we take all parameter but In you choose randomly.
- ~~Recus~~ Because when the parameter are many. Then we use Random search CV.

Ensemble Learning

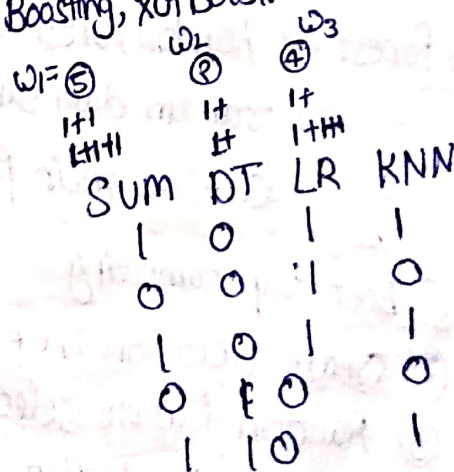
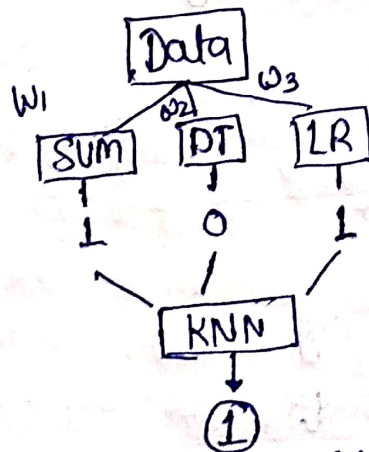
★ We hear the crowd.



• Types of Ensemble Learning.

- ① Bagging $\rightarrow$ Random forest
- ② Boosting $\rightarrow$ AdaBoost, Gradient Boosting, XGBoost.
- ③ Stacking.

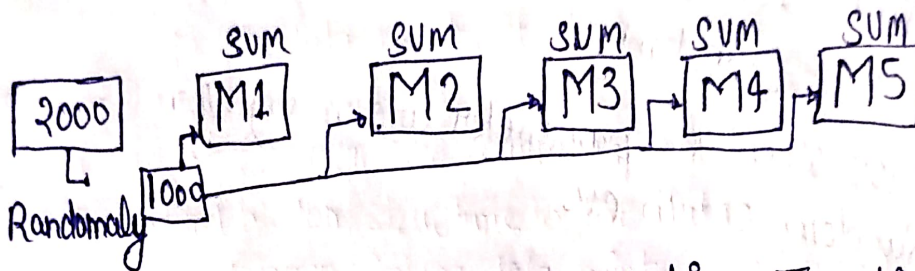
① Stacking $\rightarrow$



- Which model give you correct like KNN and so update the weight.

③ Bagging :- Bootstrap Aggregation.

- In this we make the all model is same algorithm.
- In this we choose the randomly data and the the all model.



~~Random forest~~ :- This is Bagging technique In which we train all model by the choosing randomly data from the original data set.

[2000] → every model take random [1000] data set.

③ Boosting :- Boosting trains the models sequentially. Each new model focuses on correctly the mistakes made by the previous model.

- Every new model learn from mistakes of previous one.

Random forest :- Random forest creates multiple Decision Trees using random data samples and random features then combines their prediction using majority voting.

- ① Bootstrap sampling
- ② Create Decision Tree
- ③ Random feature selection
- ④ Combine Prediction.

Classification :- voting

Regression :- Average

AdaBoost:- AdaBoost is an ensemble learning algorithm that sequentially trains weak learners and gives higher importance to misclassified sample to build strong classifier.

- ① Assign Equal weights
- ② Train the weak learner.
- ③ Identify wrong Prediction.
- ④ Updates weights
- ⑤ Train the weak learner
- ⑥ Combine all weak learner.

XGBoost:- Extreme Gradient Boosting is an advanced ensemble learning ML algorithm based on the Gradient Boosting technique.

• It build multiple Decision Trees sequentially where each new tree tries to correct the errors made by the previous tree.

- ① Start with Initial Prediction
- ② Calculate error.
- ③ Train a New Decision Tree
- ④ Sequential Training
- ⑤ Combine Prediction.

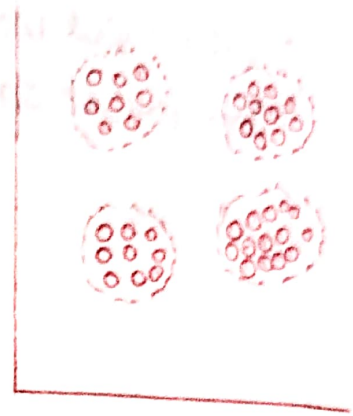
Unsupervised learning

- Unsupervised Learning is a type of Machine Learning where:
 - The data does not have labels - 'no' 'right answer' is given.
 - The model must find patterns, structure and relationship in the data by itself.
- Goals of Unsupervised Learning:
 - Find hidden structure in the data:
 - Group similar data points (clustering)
 - Reduce data dimensions while keeping important info (dimensionality Reduction)
 - Detect outliers / anomalies
 - Find associations or similarities.
- Common Techniques in Unsupervised Learning:
 - Clustering -> grouping similar data point
 - Dimensionality Reduction -> Reduce number of feature (PCA)
 - Anomaly Detection -> finding outliers in data.
- Use of Unsupervised Learning:
 - To explore and understand data structure.
 - To discover hidden pattern.
 - To reduce complexity of data.
 - To find anomalies or outliers.
 - To preprocess and improve supervised models.

e.g. -> PCA + classifier
- Use case of unsupervised learning:
 - Customer Segmentation -> group customers by behavior, spending.
 - Market Segmentation -> group market or products.
 - Anomaly Detection -> fraud detection, network security.
 - Image Compression -> Reduce image size.
 - Medical Data -> find patient groups, disease pattern.

K-means Clustering

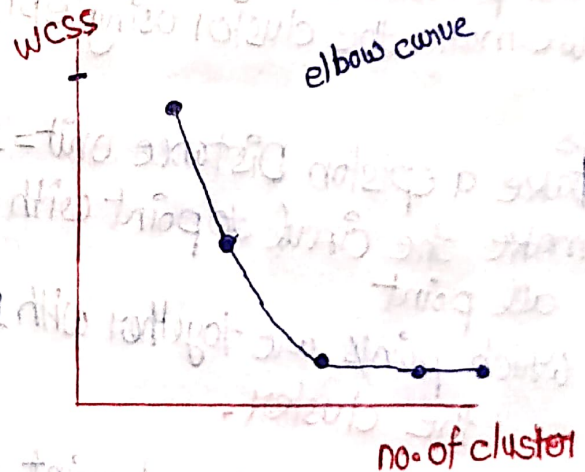
- ① Decide n Clusters $K=4$
- ② initialize centroids
- ③ Assign cluster
↳ Euclidean Distance.
- ④ Reassign the centre point
- ⑤ finish the clustering of $K=4$



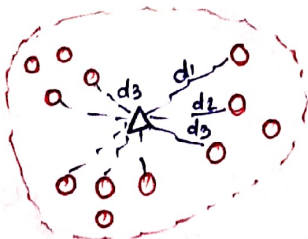
If we assign the $K=1$ then make a 1 cluster and then repeat the process for $K=4$ to find the best cluster.

- For making the clusters and decide K value you have to use Elbow methods.

WCSS -: Within clusters sum of square distance.



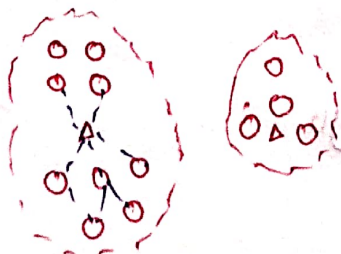
$K=1$



$$d_1^2 + d_2^2 + d_3^2 + \dots + d_n^2 = \text{WCSS}$$

is very high

$K=2$

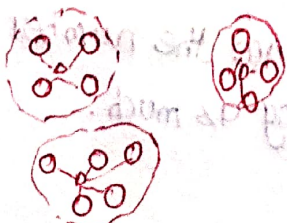


$$\text{WCSS}_1 = d_1^2 + d_2^2 + d_3^2 + \dots + d_n^2$$

$$\text{WCSS}_2 = d_1^2 + d_2^2 + d_3^2 + \dots + d_n^2$$

$$\text{WCSS} = \text{WCSS}_1 + \text{WCSS}_2$$

$K=3$



$$\text{WCSS}_1 = d_1^2 + d_2^2 + d_3^2 + \dots + d_n^2$$

$$\text{WCSS}_2 = d_1^2 + d_2^2 + d_3^2 + \dots + d_n^2$$

$$\text{WCSS}_3 = d_1^2 + d_2^2 + d_3^2 + \dots + d_n^2$$

$$\text{WCSS}_4 = d_1^2 + d_2^2 + d_3^2 + \dots + d_n^2$$

$$\text{WCSS} = \text{WCSS}_1 + \text{WCSS}_2 + \text{WCSS}_3 + \text{WCSS}_4$$

- Disadvantage of K means clustering.
 - Every time, we have to initialize the K value.
 - Outlier will move the centre point.
 - Not perfect on unconvex data.



DBSCAN

- Density Based algorithm
- non parametric algorithm.
- We make the cluster using epsilon Distance.



- Steps
- ① make a epsilon Distance unit = 1
 - ② make the circle to point with unit 1 for all point
 - ③ Which points are together with 1 unit make the cluster.

Note:- When the density of point make a clusters.

Dimensionality Reduction

- Curse of Dimensions.
 - Very difficult to find features.
 - KNN, K means, DBSCAN → Distance based algo fail in high dimensions.
 - Storage problem.
 - Model tuning time increase.
- Dimensionality Reduction is a technique used to reduce the number of input features (dimensions) in a dataset while preserving as much.

How to achieve dimensionality Reduction.

- ① Feature Selection
- ② Feature Extraction

• Most common Method of dimensionality Reduction.

- PCA (Principle Component Analysis)
- LDA (Linear Discriminant Analysis)

PCA

• PCA is a dimensionality Reduction technique that transforms a large number of correlated into a smaller number of new uncorrelated feature called principle Components, while preserving maximum information.

Why use PCA:

- Reduce the number of features.
- Reduce computational complexity
- Remove correlation b/w features.
- Reduce overfitting.
- Visualize high-dimensional data.

Working of PCA.

- ① Standardize the Data.
- ② Find correlation / covariance
- ③ Calculate Eigenvalues and Eigen Vectors.
- ④ Select Principle Components.
- ⑤ Transform the data.